

The 8085 Master Manual

Deconstructing the Machine:
A Requirement-Driven Learning Path

***Course:** Sequential Systems & Microprocessors (EE/PC/B/T/225)*

***Methodology:** Bottom-Up Component Integration*

Generated for Structural Comprehension and Architectural Mastery

Introduction: The Engineering Philosophy

We are going to build the Intel 8085 microprocessor in our minds from scratch. We will not memorize disconnected diagrams. Instead, we will start with a blank piece of silicon. As we attempt to make it perform useful work, we will encounter physical and logical limitations. To solve these limitations, we will "invent" the components of the 8085 one by one.

This document serves as your phased roadmap. It assigns exactly what you need to read from the course materials (`Microprocessor.pdf` and `Interface.pdf`) at each stage of your learning journey, mirroring a formal academic progression.

Phase I: The Physical Interface

Before internal computation can occur, a mechanism for external communication must be established. The processor **must be able to specify a location in external memory** and transfer data to or from that location.

***The Problem:** The processing core is isolated within its IC packaging. How does it communicate with the external memory chips and peripheral devices on the motherboard given a limited number of physical pins?*

1. The Address Bus (A15-A8) & Data Bus (D7-D0)

To address a standard 64 KB memory space, 16 distinct address lines (A₀ through A₁₅) are mathematically required ($2^{16} = 65,536$). Furthermore, to transfer a byte of data, 8 data lines (D₀ through D₇) are needed.

2. The Multiplexer (AD7-AD0) and ALE

The Limitation: Allocating 24 dedicated pins (16 address + 8 data) on a 40-pin DIP package leaves insufficient pins for control and power.

The Solution: Time-division multiplexing. The lower 8 bits of the address bus and the 8-bit data bus share the same physical pins, designated as **AD7-AD0**. To inform external devices whether these pins currently hold an Address (during T₁) or Data (during T₂, T₃), a dedicated control signal is introduced: the **Address Latch Enable (ALE)**. The falling edge of ALE is used to trigger an external D-latch (e.g., 74LS373), which captures and holds the lower address.

3. The Status and Control Signals (IO/M', RD', WR')

The Limitation: The processor places an address on the bus, but external devices do not know if the CPU intends to read or write, nor do they know if the address belongs to a memory chip or an I/O port.

The Solution: The introduction of the **IO/M'** pin to segregate the memory address space from the I/O address space. The active-low **RD'** and **WR'** pins are introduced to dictate the direction of data flow.

Reading Assignment for Phase I:

- **Microprocessor.pdf:**

Review the Pin Diagram and the System Bus architecture. Pay close attention to the generation of control signals (MEMR', MEMW', IOR', IOW') from the primary pins.

Phase II: Internal Anatomy

With external communication established, the focus shifts to internal data processing and execution sequencing.

***The Problem:** Data can now be fetched from external RAM. Where is this data held internally for computation, who performs the computation, and how does the system track the sequence of instructions?*

1. The Arithmetic Logic Unit (ALU) & Accumulator (A)

The ALU is the combinational logic core responsible for arithmetic and boolean operations. Because it requires inputs, a dedicated 8-bit register is inextricably linked to it: the **Accumulator (Register A)**. By convention, one operand is always sourced from the Accumulator, and the result is always deposited back into it.

2. General Purpose Registers (B, C, D, E, H, L)

Fetching data from external memory is slow. To optimize execution, six 8-bit internal registers are provided for temporary data storage. They can be concatenated into 16-bit pairs (BC, DE, HL) to handle memory addresses. The HL pair specifically functions as the default memory pointer (often denoted as 'M' in assembly).

3. The Flag Register

The Limitation: Following an ALU operation, subsequent instructions (like conditional jumps) need to know if the result was zero, negative, or produced a carry.

The Solution: A 5-bit status register composed of individual flip-flops (Sign, Zero, Auxiliary Carry, Parity, Carry) that update automatically reflecting the characteristics of the ALU's output.

4. The Program Counter (PC) & Stack Pointer (SP)

The **Program Counter (PC)** is a 16-bit register that dictates the flow of execution; it strictly holds the memory address of the *next* instruction to be fetched. The **Stack Pointer (SP)** is a 16-bit register that points to the top of a designated Last-In-First-Out (LIFO) memory area used for subroutines and interrupts.

Reading Assignment for Phase II:

- **Microprocessor.pdf:**

Study the Internal Architecture block diagram. Memorize the function of each register pair and the specific conditions that set or reset each bit in the Flag Register.

Phase III: Timing and Orchestration

A microprocessor is a synchronous sequential circuit. Every internal and external action must be precisely coordinated by a clock signal.

***The Problem:** Multiple components must interact over shared buses. Without strict temporal choreography, bus contention (data collisions) and read/write errors will occur.*

1. T-States and Machine Cycles

The fundamental unit of time is the **T-State**, which corresponds to one period of the system clock. A **Machine Cycle** is the time required to complete one basic operation (e.g., accessing memory or I/O), typically comprising 3 to 6 T-states. An **Instruction Cycle** consists of one to five Machine Cycles.

2. The Opcode Fetch Cycle

The first cycle of any instruction is the Opcode Fetch.

- T₁: PC places address on the bus; ALE pulses high.
- T₂: RD' goes low; memory is selected.
- T₃: Opcode is placed on the data bus and latched by the CPU.
- T₄: The CPU internally decodes the opcode (bus is idle).

3. The READY Pin and Wait States

The Limitation: The CPU operates at 3 MHz, but a connected ROM chip has an access time of 500 ns. The CPU will read the data bus before the ROM has placed valid data on it.

The Solution: The **READY** input pin. The CPU samples this pin during T₂. If READY is LOW, the CPU suspends sequencing and inserts a **Wait State** (T_{WAIT}), holding all signals stable until the slow device drives READY HIGH.

Reading Assignment for Phase III:

- **Microprocessor.pdf:**

Analyze the Timing Diagrams for Opcode Fetch, Memory Read, and Memory Write. Understand the structural differences between them.

Phase IV: Address Decoding and Interfacing

Connecting a memory chip to the system buses is insufficient; specific logic must be designed to ensure that only the correct chip responds to a given 16-bit address.

***The Problem:** When the CPU outputs address 2000H, multiple connected memory chips might attempt to respond simultaneously if they only monitor the lower address lines.*

1. Chip Select (CS') Logic

Every peripheral or memory IC possesses an active-low Chip Select (CS') pin. The IC remains in a high-impedance state until this pin is asserted.

2. Absolute vs. Partial Decoding

To assert the correct CS', the higher-order address lines must be decoded.

- **Absolute Decoding:** Every single unused address line is routed into the decoding logic (e.g., using a 74LS138 3-to-8 decoder). This maps the chip to one, and only one, logical address range.
- **Partial Decoding:** Some address lines are left unconnected (treated as "Don't Cares") to simplify hardware.

The Consequence: This leads to **Image Addresses** (or foldback addresses), where the same physical chip responds to multiple different logical addresses, wasting the 64 KB address space.

3. I/O Interfacing Methodologies

Peripheral devices can be interfaced using **Memory-Mapped I/O** (treating the device as a 16-bit memory address, utilizing $IO/M' = 0$ and standard LDA/STA instructions) or **I/O-Mapped I/O** (utilizing the isolated 8-bit port address space, $IO/M' = 1$, and IN/OUT instructions).

Reading Assignment for Phase IV:

- **Interface.pdf:**

This document is critical. Study the logic gate implementations for partial decoding and the 74LS138 schematics for absolute decoding. Master the calculation of base and image addresses.

Phase V: Assembly Language Programming (ALP)

With the hardware architecture complete, the system must be directed through software to perform algorithms.

***The Problem:** How do we utilize the basic instruction set to construct complex logic, such as loops, conditional branching, and mathematics beyond simple addition?*

1. Addressing Modes

The methodology by which an instruction specifies its operand:

- **Immediate:** Data is within the instruction (`MVI A, 05H`).
- **Register:** Data is in an internal register (`MOV A, B`).
- **Direct:** The 16-bit memory address is specified (`LDA 8000H`).
- **Indirect:** The address is held in a register pair, usually HL (`MOV A, M`).

2. Subroutines and the Stack

The Limitation: Repeating a block of code (like a time delay) in multiple places consumes excessive ROM space.

The Solution: The **Subroutine**. The `CALL` instruction pushes the current Program Counter onto the Stack and jumps to the subroutine. The `RET` instruction pops the Stack back into the Program Counter, resuming the main program.

3. Algorithmic Structures

Because the 8085 lacks a hardware multiplier or divider, these operations must be constructed using loops (e.g., repeated addition). Software delays are constructed by loading a large number into a register pair and decrementing it to zero in a loop, calculating the exact T-states to achieve a specific temporal delay.

Reading Assignment for Phase V:

- **Microprocessor.pdf:**

Review the Instruction Set classification. Study the provided Assembly Language Programs (ALPs): Block Transfer, Multiplication, Largest/Smallest Array element, and BCD conversions.

Phase VI: The Interrupt Architecture

A sequential program cannot efficiently monitor external, asynchronous events without wasting processor time in polling loops.

***The Problem:** The processor is executing a long algorithm. An external sensor detects a critical fault. How is the processor forced to suspend its current task and respond immediately?*

1. Hardware Interrupts and Priority

The 8085 provides five hardware interrupt pins, prioritized to handle simultaneous requests: TRAP (Highest), RST 7.5, RST 6.5, RST 5.5, and INTR (Lowest).

2. Vectoring

Upon acknowledging an interrupt, the processor must jump to an Interrupt Service Routine (ISR). For **Vectored** interrupts (TRAP and the RSTs), the CPU internally calculates the vector address (e.g., $6.5 \text{ times } 8 = 0034\text{H}$). For the non-vectorized INTR, the external device must place a `CALL` instruction on the data bus.

3. Masking via SIM and RIM

The Limitation: During critical code execution, low-priority interrupts must not disrupt the processor.

The Solution: **Masking.** The programmer uses the **SIM (Set Interrupt Mask)** instruction to selectively disable RST 5.5, 6.5, or 7.5. The **RIM (Read Interrupt Mask)** instruction is used to read the status of pending interrupts that were triggered while masked. TRAP is non-maskable; it guarantees an immediate response regardless of software state.

Reading Assignment for Phase VI:

- **Microprocessor.pdf:**

Study the Interrupt mechanism. Memorize the vector address calculations and the specific bit-formats for the Accumulator when executing the SIM and RIM instructions.